Инженерингът на изискванията е систематичен процес на **идентифициране, анализиране, документиране и валидиране** на функционалностите и ограниченията на даден софтуер. Докато програмирането отговаря на въпроса „как“, инженеринг на изискванията отговаря на въпроса **„каква система трябва да се реализира“**.

Цели и задачи:

- Идентифициране и решаване на проблеми.
- Дефиниране на обхвата на системата.
- Свързване на различните заинтересовани лица (ЗЛ) – клиенти, потребители, разработчици; те идентифицират проблеми които се нуждаят от решения
- Осигуряване на основа за проектиране, тестване и приемане на продукта.

Основни етапи на процеса:

1. **Извличане (Elicitation):** Търсене и откриване на изисквания от ЗЛ и други източници.
2. **Анализ и преговори:** Уточняване на изискванията и решаване на конфликти между тях.
3. **Документиране (Специфициране):** Записване на изискванията в структуриран документ.
4. **Валидиране:** Проверка дали изискванията точно отразяват нуждите на клиента.
5. **Управление:** Паралелна дейност за контрол на промените и проследимост.

Изискванията се разделят на няколко основни категории, за да се улесни тяхното управление и реализация:

Категория	Описание	Примери
Функционални (ФИ)	Какво трябва да прави системата (действия, функции). Документират се от три перспективи: - Перспектива на данните — как данните се използват и структурират - Функционална перспектива — условия за ползване, обработка на данни, резултати - Перспектива на поведението — динамиката на проявление (реакция на събития)	„Системата позволява регистрация на потребител.“
Нефункционални (НФИ)	Поставят ограничения върху продукта и процеса на разработка. Отнасят се до системата като цяло, не до конкретна функционалност.	Производителност, сигурност, надеждност, използваемост.
Бизнес изисквания	Цели на организацията и бизнес процеси, които системата трябва да поддържа.	„Намаляване на времето за обработка на поръчки.“
Системни изисквания	Технически изисквания и ограничения, свързани с хардуера и софтуера.	Хардуерни ограничения, софтуерни зависимости.
Домейн изисквания	Произтичат от областта на приложение, а не от клиентски нужди. Могат да бъдат ФИ или НФИ; произлизат от природни закони или индустриални стандарти.	Банкови регулации, физични закони, медицински стандарти.

```
Извличане → Анализ + Преговори → Документиране → Валидиране
                              ↑_____ Управление на изискванията (паралелно) _____↑
```

Дейност	Цел
Извличане	Търсене, откриване, обобщаване на изисквания от заинтересованите лица
Анализ	Открива незавършеност, несъгласуваности; осъществимост
Документиране	Записва изискванията в документ
Валидиране	Проверява дали изискванията дефинират точно желаната система
Управление	Управлява промените в изискванията и връзките между тях

Извличане на изискванията

- **Интервюта и анкети:** Пряка комуникация със ЗЛ за разбиране на техните нужди.
- **Сценарии и Случаи на употреба (Use Cases):** Примери от реалния живот за взаимодействие със системата.
- **Наблюдение (Етнография):** Анализ на потребителите в реалната им работна среда.
- **Прототипиране:** Създаване на ранна версия на системата за събиране на обратна връзка.
- **Мозъчна атака (Brainstorming):** Екипна работа за генериране на идеи и алтернативи.

Анализ на изискванията

- **Проверка за необходимост и осъществимост:** Реалистични ли са изискванията спрямо бюджета и технологиите?
- **Матрица на зависимостите:** Откриване на конфликти и припокривания между изискванията.
- **Приоритизиране:** Използване на методи като **MoSCoW** (Must, Should, Could, Won't).

Валидиране на изискванията

- **Прегледи (Reviews):** Систематичен ръчен анализ на документацията.
- **Генериране на тестови случаи:** Ако едно изискване не може да бъде тествано, то е лошо дефинирано.

Документирането превръща събраната информация в техническа спецификация.

Документ на изискванията съдържа:

- Изисквания на потребителите — разбираеми без техническа подготовка
- Системни изисквания — технически детайли; може да са основа за договор

За всяка функционалност трябва да се документират 3 аспекта:

1. **Данни** — как се използват и структурират
2. **Функции** — условия, обработка на входа, очаквани резултати
3. **Поведение** — динамика при събития

Моделите помагат за визуализиране на системата от различни перспективи:

- **Модел на потока на данните (DFM/DFD):** Показва как информацията се движи и трансформира в системата. => Елементи: Процеси, Външни субекти, Хранилища на данни.

Елементи:

Елемент	Нотация	Описание
External Entity	Правоъгълник	Външен потребител/система; извън обхвата
Process	Кръг/овал	Трансформира или манипулира данни
Data Flow	Стрелка	Пренася данни между елементи
Data Store	Двойна линия	Хранилище на данни (файл, БД)

```
[User] --Order--> (Process Order) --Save--> =|OrderDB|=
                        |
                   --Confirm--> [User]
```

Нива: Ниво 0 (Context diagram — цялата система като 1 процес), Ниво 1, Ниво 2...

Компоненти:

- **Състояние** (State) — ситуация, в която системата изчаква или обработва
- **Събитие/Действие** (Event/Action) — външно/вътрешно или времево
- **Преход** (Transition) — промяна на състоянието при събитие

```
[Idle] --login()--> [Authenticated] --logout()--> [Idle]
[Authenticated] --order()--> [Processing] --confirm()--> [Authenticated]
```

- **Семантичен модел (ERD):** Представя обектите (същностите) в базата данни, техните атрибути и връзки. =>

Компоненти:

- **Същност (Entity)** — обект или дейност (правоъгълник)
- **Атрибут** — свойство на същността (елипса)
- **Връзка (Relationship)** — ромб

Кардиналност: 1:1, 1:N, N:1, N:M
Опционалност: задължителна / незадължителна

```
[Customer] ---places--- [Order] ---contains--- [Product]
              1:N                    N:M
```

- **Обектно-ориентирани модели (Клас диаграми):** Обединяват данни и функционалност чрез обекти, класове и наследяване.
  - Основни концепции: Обект, Клас, Операции, Капсулиране, Наследяване, Съобщения.
  - UML диаграми: Class Diagram, Sequence Diagram, Use Case Diagram.
- **Формални модели:** Използват математическа нотация за максимална точност (синтаксис, семантика, връзки).
  - Компоненти: Синтаксис (нотация), Семантика (обекти), Връзки (правила).
  - Примери: Z нотация, VDM, Alloy.